BMC-K4510

USER'S AND PROGRAMMER'S GUIDE

a fantasy 8/16-bit computer

Draft version: alpha-0.2+88 · CPU 45GS10 · VICKY · 4 reSIDs · 256 MB RAM

Date: 26.08.2026

Contents

Part I

User's Guide

Chapter 1

The Machine

The BMC-K4510 is a fantasy computer: a machine that never existed, built the way 1985 might have built it with no budget committee in the room. It is not an emulation of any real computer. Its CPU, video chip, sound, and operating software are its own; the parts it borrows — a 6502-family instruction set, the SID sound chip — it borrows openly and then outgrows.

1.1 What is in the box

- **CPU: the 45GS10** — the MEGA65's 45GS02 instruction set (a 4510 with the Q pseudo-register and 32-bit flat addressing) plus this machine's own memory management: bank registers, a far-call gate, and RAM under the ROM. It runs at 40.5 MHz.

- **Memory: 256 MB**, flat, 28-bit. The CPU sees 64 KB at a time; everything else is one instruction away.

- **Video: VICKY** — 640×480, 256 colours from a 24-bit palette, four layers, 128 sprites, a blitter that draws lines and filled triangles, and a display-list coprocessor named SHEILA. Smaller modes (640×240, 320×240, 320×200, 160×200) are drawn into the same glass, doubled and centred, so the raster is always 480 lines however few of them the picture uses.

- **Sound: four SID chips** (reSID 6581) and a floating-point MATH unit the BASIC leans on.

- **A system ROM**: a shell with directories, a Wozmon-style monitor, and EhBASIC 2.22 extended with graphics and sound — with three more languages one word away: BBC BASIC on the Tube, Forth, and CP/M.

1.2 Getting one

On the **desktop** (Linux), three lines build the whole machine:

```
git clone https://github.com/mlongval/bmc-k4510
cd bmc-k4510 && ./setup.sh
./k4510
```

`setup.sh` installs the compilers and SDL2, builds everything, and runs the machine's ten test suites.

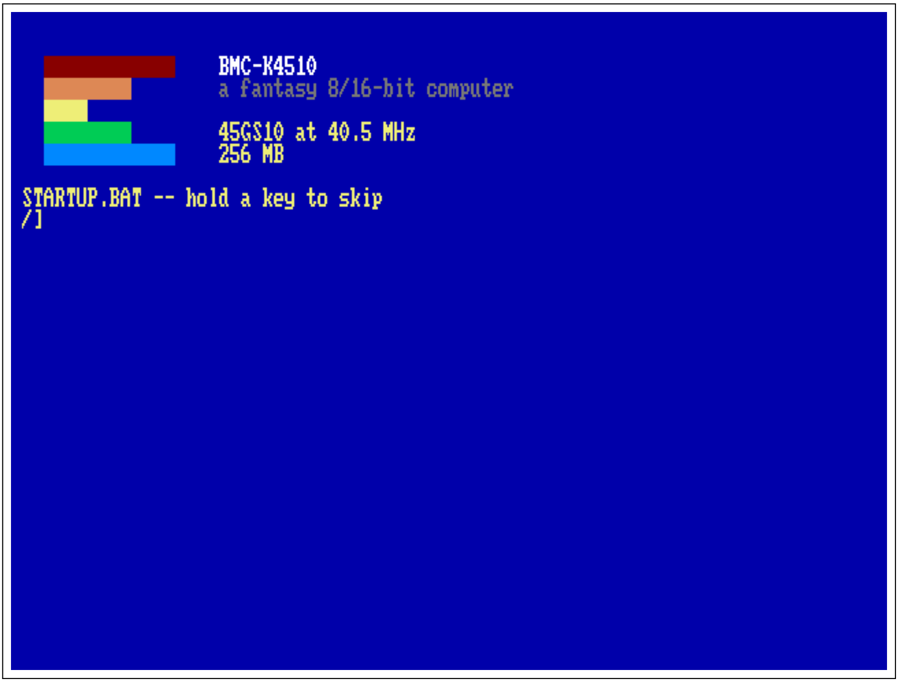
On the **Raspberry Pi 3B+**, download the SD-card zip from the project's releases page and put it on a card — either unzip it onto the card's root yourself, or let the script do it:

```
./install-sd.sh bmc-k4510-pi3.zip /media/you/CARD
```

Use an SDHC card — 4 to 32 GB, specifically. SDHC ships FAT32 from the factory and just works. Older plain SD cards (2 GB and under) do *not* boot — we tested. Bigger SDXC cards (64 GB+) ship exFAT and will not boot until reformatted; `./install-sd.sh zip /dev/sdX --format` does that (and erases the card). The machine needs about 5 MB, so the smallest SDHC card sold is plenty.

1.3 First boot

Switch it on (on the desktop: `./k4510`; on the Raspberry Pi 3B+: insert the card and power up). The machine greets you in yellow on blue:



The boot screen: the hourglass, the name, the speed, the memory. Everything else is one `INFO` away.

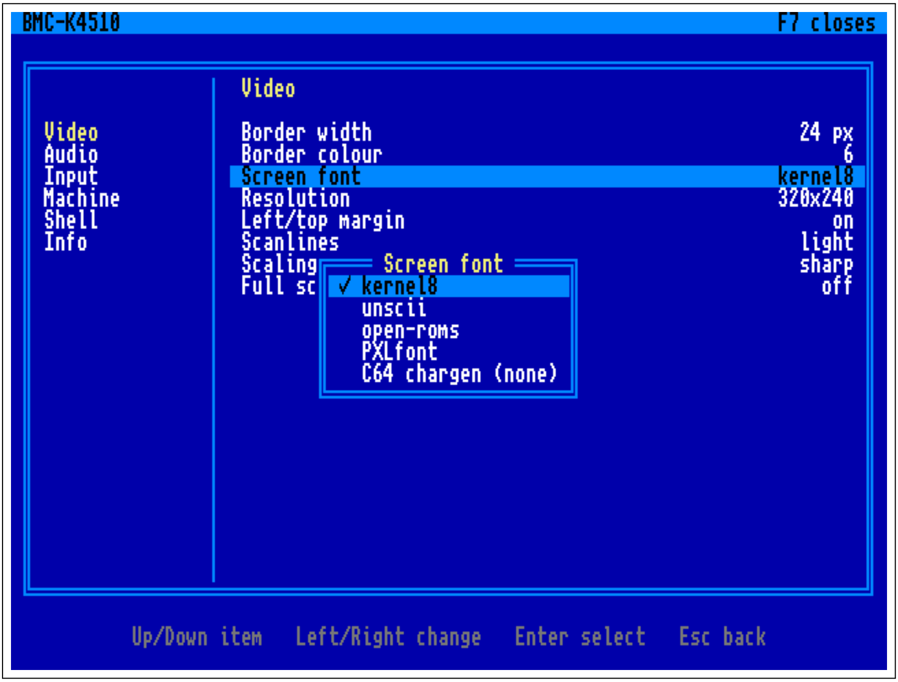
The `/]` at the bottom is the shell prompt — the `/` is the directory you are in. Type `HELP` for the commands, `INFO` for the machine's full self-description, and `DIR` to see your files.

> **Keys on the desktop:** Escape is RUN/STOP (stops a BASIC program), Ctrl-C is STOP, Shift+Escape quits the emulator. Reset is a chord, the way it was Commodore+Restore on the C64: Super+PageUp (the Windows or Command key and PageUp) — one key cannot do it by accident. F7 opens the menu.

1.4 The F7 menu

F7 (unshifted; Shift+F7 still reaches BBC BASIC) freezes the machine and takes the whole screen, in the manner of the C64 Ultimate's: the categories down the left, the settings of the chosen one on the right, and a line at the foot saying what the keys do in whichever pane holds the cursor. Sound stops, and closing the menu resumes exactly where it froze — the machine is not disturbed, only hidden. The menu draws with its own font and never touches the machine's memory, so it opens even when a program has wrecked the screen, and being opaque it reads the same whatever was there.

Up and Down move; Enter or Right crosses from the categories to the settings; Left and Right step a value; Escape goes back a pane, and closes at the categories; F7 closes from anywhere. A setting with a list of choices opens that list over the panes, and *applies as the cursor passes each one* — the screen font swaps under the menu so you can see it — with Escape putting back whatever was there when the list opened.



The F7 menu: categories on the left, the Video page on the right, the screen-font list open over them.

Video border width and colour around the picture; the screen font (*kernel8*, *unscii*, *open-roms*, *PXLfont* — a live swap of the chargen at `$010000`, the two PETSCII fonts rearranged into ASCII order, and *C64 chargen* if you have put a Commodore character ROM in `/SYSTEM/chargen.bin` — the entry says so when you have not); *scanlines*, a dark line between each of the machine's, in three strengths — the picture is drawn at twice the height and every second row dimmed, so it is exact rather than an overlay, and it costs about a third of a millisecond a frame; *scaling*, which is how the picture is fitted to the window: *sharp* (each pixel a hard square), *soft* (smoothed), or *sharp-fit* (a whole-number multiple, then smoothed for whatever is left over — crisp text at any window size, which neither of the others manages); full screen.

The figures in this book are taken with the effects off, whatever the settings say, so that what you see here is the machine's own output.

Audio volume.

Input the reset chord (Super, Ctrl or Alt + PageUp, or Ctrl+Alt+Del); which key opens the menu (F7, F8, F11 or Pause).

Machine *save state* and *load state*, four slots each (the whole machine — CPU, every used page of the 256 MB, the banks and MAP, VICKY, the devices, JIM — to `k4510-slotN.k4s` beside the settings file; the row shows when the slot was written; the Tube co-processor is not in the file and is stopped by a load); reset; power cycle (memory cleared, ROM reloaded); stop the Tube co-processor; quit.

Shell whether an unknown word at the prompt may run a CP/M `.COM` from drive A: (Chapter 6). Off to begin with, on purpose: `D` typed for `DIR` should not start a Z80 program. It changes only the guess — a command that names `CPM` outright, an alias among them, works either way. This is the first setting the machine itself reads: the host puts the menu's switches in `$D521` and the ROM looks there.

Info version, ROM, files, host.

Leaving the menu writes the settings to `k4510.cfg` beside `fs/` (on the Pi, `SD:/k4510/k4510.cfg`) — a plain `key = value` file you may edit; comments and keys it does not know are kept. Adding a setting is one row in `core/ui/settings.c`, one row in the menu tree in `core/ui/menu.c`, and the place that reads it; the same code runs on the Pi, where the C64 keyboard's F7 opens it too.

Chapter 2

The Shell

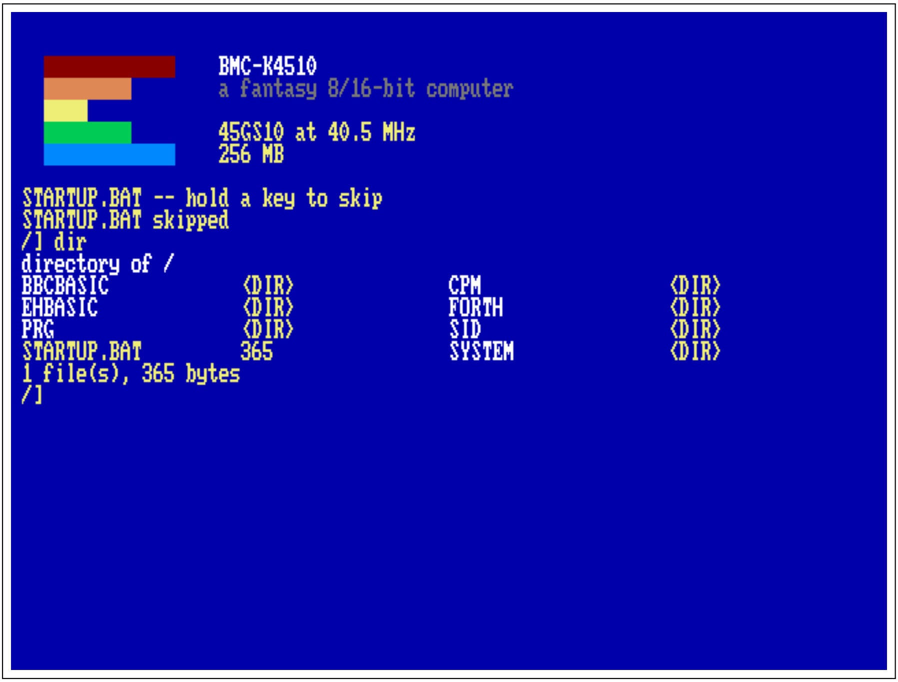
The shell is what the machine boots into. It is a file manager, a program launcher and a front door to the monitor, and every command here also works from BASIC with a * in front. Type `HELP` for the whole command set on one screen — the text it prints is itself a file on disk, a hidden one called `/.HELP`.

2.1 Files and directories

Files live on the host (the `fs/` directory on the desktop, the `k4510/fs/` folder of the SD card on the Pi). The layout is one directory per language: machine code in `/PRG`, the BASICs in `/EHBASIC` and `/BBCBASIC`, Forth in `/FORTH`, CP/M's drives under `/CPM`, music in `/SID` — and a bare name is searched across the language directories too, so you rarely need a path.

```
DIR                CD EHBASIC        CD ..
MKDIR MYDIR        RM OLD.TXT        RMDIR MYDIR
TYPE README.TXT    RENAME OLD NEW    CP FROM TO
LOAD name [addr]   SAVE name from.to
XD name            a hex dump (HEX works too); Esc stops it
```

Names that begin with a dot are hidden; `DIR A` shows them. `DIR` also takes a directory to look in, or a pattern to match: `DIR PRG` lists that folder without going there, `DIR *.PAS` and `DIR ?.TXT` match here (* any run of characters, ? any one), and `DIR A *.BAS` means both at once.



`DIR`: directories first in white, then files with their sizes, two columns.

2.2 Running things

`RUN balls.prg` runs a program; so does `RUN balls`, and so does plain `BALLS` — an unknown word is tried as a program on disk before the shell gives up, *with its arguments*, the way REXX did it on the mainframes. A program reads its arguments through the `ARGS` system call; `SAY` is the demonstration:

```
SAY HELLO FROM THE DISK
```

prints `HELLO FROM THE DISK` — there is no `SAY` command, only a `say.prg` in `/PRG`. Try `SIDPLAY`, the SID jukebox; `EDIT` and `VI` are the editors (Chapter 8); and four words open whole languages: `EHBASIC` (Chapter 3), `BBC` (Chapter 4), `FORTH` (Chapter 5) and `CPM` (Chapter 6).

2.3 SWAP: running one thing from inside another

A program is loaded where the last one was, so starting one from inside a BASIC would ordinarily flatten the program you were writing. `SWAP command` puts the caller away first — the whole of the CPU's memory, and the screen — runs the command on a clean machine, and gives the caller back untouched:

```
*SWAP EDIT DEMO.BAS
```

from EhBASIC edits a file and returns to your program, variables and screen as they were. One level deep; the thing being run must be an ordinary program, and a BASIC's far memory is not disturbed because nothing else touches it. BBC BASIC needs none of this: it lives on the co-processor, out of reach of anything running here.

2.4 Aliases

`ALIAS` gives a name to a line.

```
ALIAS                   list them
ALIAS LL DIR A          define one
ALIAS LL                nothing after the name: remove it
```

Whatever you type after the alias is added to the end, so a definition takes arguments. Aliases are tried *last*, after every other rule, so one can never hide a real command: `ALIAS DIR ECHO no` is accepted and `DIR` still lists the directory. They live in memory — in a sideways bank, not in the 64 KB — so they survive a reset but not a power cycle, which is what `/STARTUP.BAT` is for.

2.5 The network

A URL is a file name. Anything the shell reads — `TYPE`, `LOAD`, `CP`, `RUN`, and the bare-word rule above — accepts `http://` or `https://` in place of a name, and the machine fetches it:

```
TYPE https://raw.githubusercontent.com/mlongval/bmc-k4510/master/README.md
CP http://example.org/game.prg game.prg
RUN http://example.org/game.prg
```

EhBASIC's `LOAD` and BBC BASIC's `LOAD` on the Tube take URLs the same way. The idea is the C64's Meatloaf cartridge, where a disk name could be an address on the internet; here it costs the ROM nothing, because the ROM never looks at a name — the host does the fetching. A typed line is 95 characters, and so is the longest URL.

A `tnfs://` URL is more than a file: it is a place. TNFS is the little UDP file protocol the FujiNet and Meatloaf servers speak, and the machine's current directory can be on one of them:

```
CD tnfs://fujinet.online/
DIR
CD CBM
DIR
CD -
```

`DIR` lists the server, a bare name loads from it (so `RUN`, and the bare-word rule, run programs straight off the internet), `CD ..` climbs, `CD -` comes home. The prompt shows where you are. The server is read-only from here.

For programs that want a live connection there is the *N: device* at `$D900` (FujiNet's name for it): four channels, each a URL opened for reading and writing — `tcp://host:port` for a connection, `http://` for a page. `TELNET host port` is the demonstration, a `telnet.prg` in `/PRG`: what you type goes out, what arrives is drawn by JIM, the terminal (Chapter 4), so a BBS gets its ANSI colours and CP437 art and the cursor and function keys go out as VT sequences; F12 hangs up (Escape belongs to the far end). The register map is in Chapter 10 and `core/net.h`. On the Pi the network is the Ethernet port (DHCP); without a cable the device answers "not fitted" and URLs are simply absent names. The Pi has no TLS, so `https://` works on the desktop only.

2.6 CAPSLOCK

`CAPSLOCK` toggles a caps-lock: while it is on, letters read from the keyboard come up uppercase, so a language whose keywords are uppercase — BBC BASIC, EhBASIC — needs no Shift held. Digits and symbols are untouched, so it is a caps lock and not a shift lock. `CAPSLOCK ON` and `CAPSLOCK OFF` set it explicitly. It works from inside a BASIC through the * escape: `*CAPSLOCK` in BBC BASIC or EhBASIC.

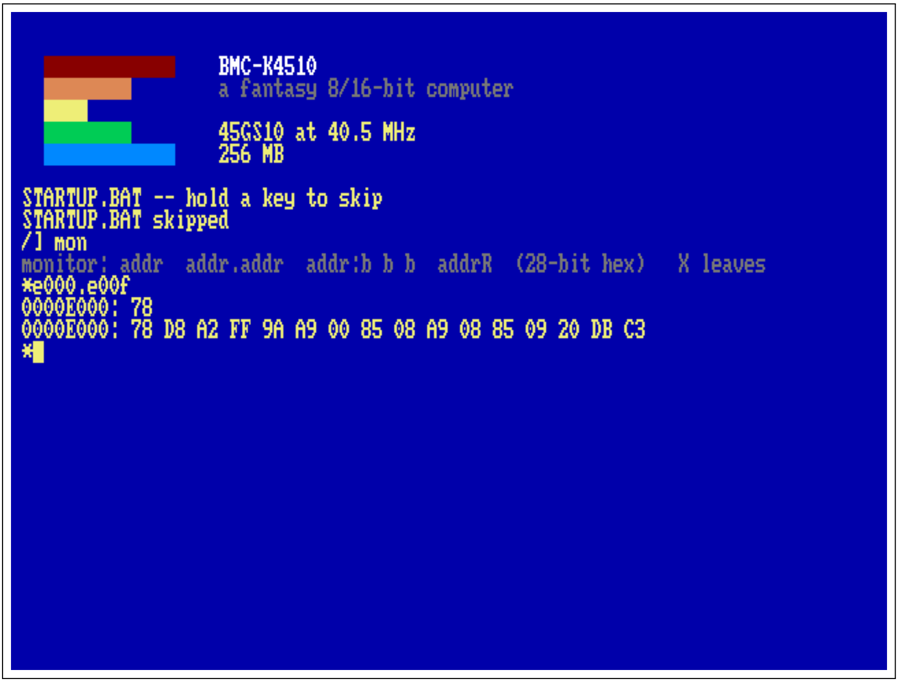
2.7 Scripts and the boot file

`EXEC name` runs a text file as shell commands, one per line. A line whose first character is # is a comment, and a blank line is ignored. At power-on the machine runs `/STARTUP.BAT` as such a script, if it exists — half a second's grace, and any key held or typed skips it without the keystroke being lost. It is the place to put your aliases; `/SYSTEM/STARTUP.SAMPLE` is one to copy from.

2.8 The monitor

`MON` (old-timers may type `WOZ`) opens the machine monitor at a * prompt; its grammar is Wozmon's, with 28-bit addresses:

```
*E000.E00F          examine a range
*0300:A9 41 60      store bytes
*0300R              run from $0300
*X                  leave
```



The monitor examining the ROM. `MON E000.E00F` runs one line without entering the prompt.

2.9 Housekeeping

`INFO` is the machine's self-description; `TIME` the clock; `MODE` and `COLOR` set the text screen — `CLS` clears it and `CLG` clears the bitmap over it, whoever drew it — both reach a BASIC through the * escape, which is how you tidy up after a demo that left its picture behind; `FILL` and `COPY` work on memory; `HUSH` silences all four SIDs at once; `LS` is `DIR` for fingers that have used Unix too long.

`MODE` on its own says where you are; `MODE n` moves. The second number is the margin: 1 (the default) keeps a one-cell gap at the top and the left, 0 uses every cell.

MODE	pixels	text	
0	640×480	80×60	the whole glass
1	640×240	80×30	the machine starts here
2	320×240	40×30	
3	320×200	40×25	the C64's geometry
4	160×200	20×25	four screen pixels each

The F7 menu, under Video, has the same two knobs: *Resolution* shows what the machine is actually in — it follows a `MODE` you type — and choosing another asks the ROM to perform it, because the console's geometry belongs to the ROM and not to the host. *Left/top margin* is the second parameter.

The menu offers only the first three. MODE 3 and MODE 4 are for a game or a language that wants the pixels: 40×25 and 20×25 are not a shell, and a machine you have steered into 20 columns from a menu is a machine you then have to steer out of. They are a `MODE` command away, the menu shows one when you are in it, and neither is ever written to `k4510.cfg` — what is saved is never smaller than 320×240.

The picture is always painted into 640×480: a 200-line mode gets 40 blank lines above and below it, and a 320- or 160-wide one has its pixels doubled or quadrupled sideways. Raster lines and SHEILA's display list count the glass, 0–479, not the mode — so in MODE 3 the first line of your picture is raster line 40.

2.10 When STARTUP.BAT is the problem

`/STARTUP.BAT` runs at power-on, and a bad one runs at every power-on. The machine says so at the banner:

```
STARTUP.BAT -- hold a key to skip
```

and gives you half a second. Hold any key and it says `STARTUP.BAT skipped` instead of running it. If that moment is hard to catch, switch it off in the F7 menu under Shell: *Run STARTUP.BAT*. That setting is the host's, kept in `k4510.cfg`, so it survives a power cycle and no wedged program in the machine can take it away from you. Boot clean, fix the file, switch it back on.

2.11 When something goes wrong

`DUMP a note` writes the whole machine state — registers, screen, the last 4096 instructions, your last keystrokes — to a numbered file on the host, for a human (or their AI) to read later. `DUMP ON` does it automatically every fifteen seconds.

Chapter 3

EhBASIC

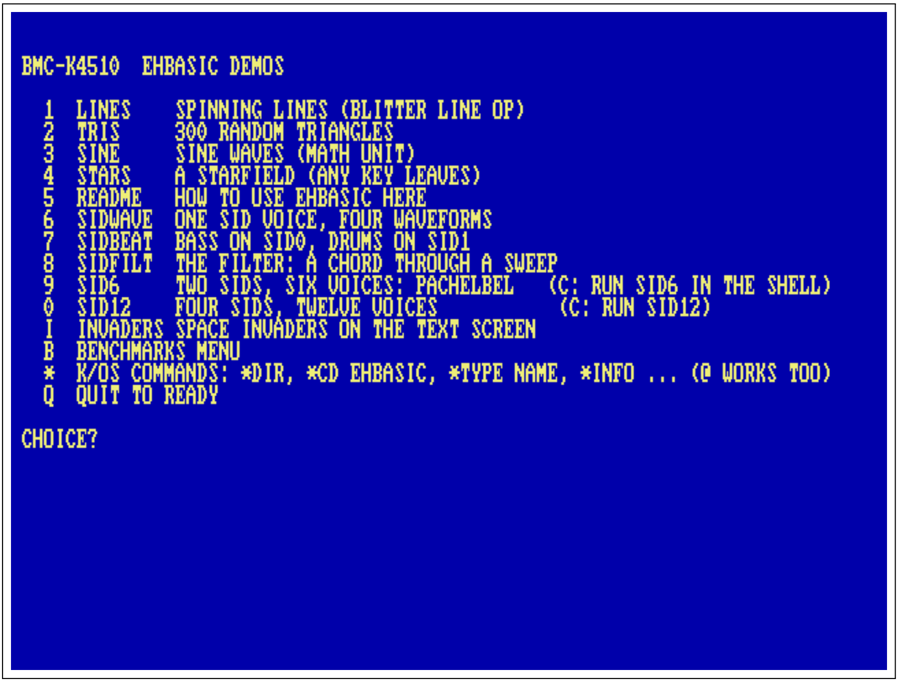
The machine speaks EhBASIC 2.22 — Lee Davison's Enhanced BASIC — with this machine's additions: graphics statements that ride the blitter, sound on four SIDs, floating point on the MATH unit, and an escape hatch to the shell.

```
RUN EHBASIC
```

It comes up ready: the banner, `47103 Bytes free` — more than twenty-one C64s more than a C64 — and a `Ready` prompt — the traditional `Memory size ?` question is gone; the machine knows.

3.1 Ten minutes of it

```
RUN "DEMOS.BAS"
```



The demo menu. A key picks an entry; each demo returns here when it ends.

The menu chains: `RUN "name"` (or `LOAD` inside a running program) loads another BASIC program and runs it — that is how one BASIC program hands over to another, exactly as on a C64.

3.2 The machine's own statements

```
GRAPHICS 2          640x480 bitmap over the text
PLOT X,Y,C          LINE X1,Y1,X2,Y2,C
TRI X1,Y1,X2,Y2,X3,Y3,C
PALETTE I,R,G,B     GCLS            GRAPHICS 0
```

Colours 0–15 belong to the text screen; demos use 16 and up. `GRAPHICS 0` puts the text mode and its palette back, whatever the program changed.

3.3 The * escape

Any shell command works from BASIC with * in front — `*DIR`, `*CD EHBASIC`, `*MON`, `*DUMP a note` — and `*BYE` leaves BASIC for the shell (`@` is accepted too, a survivor from an earlier machine). Escape or Ctrl-C stops a running program (and silences the SIDs); the program's variables survive for `PRINT`-style post-mortems.

3.4 Editing the program in VI

`*VI` and `*EDIT`, with nothing after them, edit the program that is in memory:

```
10 PRINT "HELLO"
*VI
```

BASIC writes the program to a temp file, opens the editor on it, and reads it back when you leave — so what you type in the editor is what you `LIST` afterwards. Give either one a file name and nothing special happens: `*VI notes.txt` is the ordinary * escape, and the editor edits that file.

The trip out and back is not free. The editors load where the interpreter lives, so the shell command underneath is `SWAP`, which puts all 64 KB and the screen aside first and restores them afterwards. And because the program is written out and read back, *variables do not survive* — this is an immediate-mode thing, like `LOAD`. The temp file (`EDITTMP.BAS`) is left behind, which is occasionally a useful accident.

Why the first line is blank: `SAVE` starts its output with a newline, so the editor opens on an empty line 1 with the program below it. Harmless — BASIC ignores it on the way back in — but it is why the line count is one more than you expect.

Chapter 4

The Tube: BBC BASIC

The BBC Micro's most elegant idea was the Tube: a fast port through which a *second processor* — another CPU with its own memory — could take over the computation while the Beeb kept the keyboard, the screen and the discs. The BMC-K4510 has a Tube of its own at `$D800`, and the first thing fitted to it is Richard Russell's BBC BASIC, running on the host machine with a flat 256 MB of its own.

```
BBC
```

You get the > prompt of a BASIC with real power behind it, and it is *fast* — the co-processor is not cycle-matched to 1985:

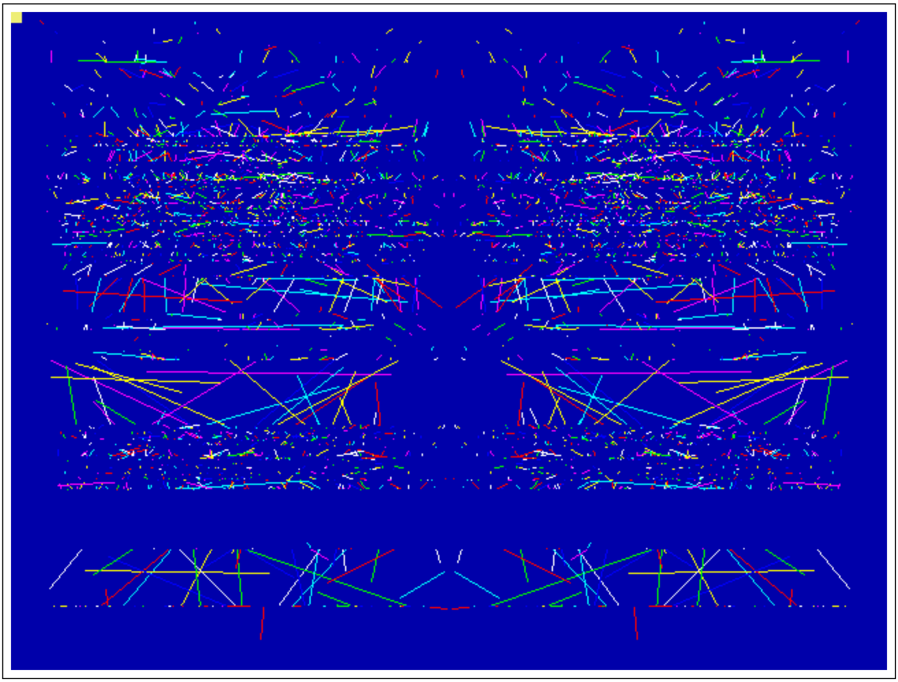
```
HIMEM=PAGE+250*1024*1024
DIM space 200*1024*1024
```

both just work. Type `*QUIT` to hand the console back to the shell.

4.1 Graphics and sound

The console cannot show what it cannot see, so the Tube speaks up: MODE, GCOL, PLOT, MOVE, DRAW, CIRCLE and the palette arrive at the machine as escape sequences, and the Tube's gatekeeper (the *Tube ULA*) executes them on the VICKY blitter. `SOUND` and `ENVELOPE`-less music go the same way, onto the machine's four-channel sound sequencer and out through the SIDs — the classic Beeb `SOUND chan,amp,pitch,dur`, queued per channel like the original.

```
LOAD "BBCBASIC/KALEID.BBC"
RUN
```



KALEID.BBC: MODE 2 kaleidoscope, drawn by BBC BASIC on the Tube, painted by VICKY.

The /BBCBASIC directory carries a period programme selection: KALEID, CIRCLES, ROSES, MOUNTAIN, BOUNCE, CLOCK (a live analogue clock face) and TUNE — Frère Jacques as a three-voice round, the sound sequencer's party piece.

4.2 Sprites

VICKY has 128 hardware sprites, and BBC BASIC reaches them the way RISC OS reached its own: VDU 23,27 selects and shapes, PLOT &ED places. There is no sprite file. A sprite is *captured* from the bitmap after BBC BASIC has drawn it with the words it already knows:

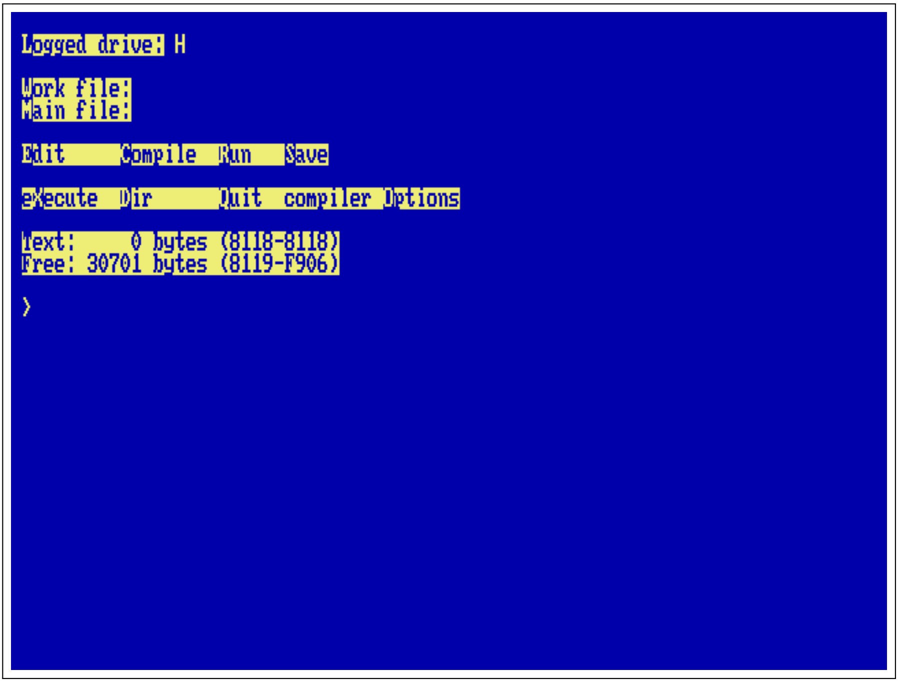
```
MODE 2
GCOL 0,1:CIRCLE FILL 40,40,30
MOVE 8,6:VDU 23,27,1,0,32,32|
CLG
PLOT &ED,600,500
```

`VDU 23,27,0,n|`	select sprite n (0–127) for PLOT
`VDU 23,27,1,n,w,h|`	capture n, $w \times h$ pixels (8/16/32/64), bottom-left at the graphics cursor
`VDU 23,27,2,n|`	hide n
`VDU 23,27,3,n,f|`	flip: bit 0 horizontal, bit 1 vertical
`VDU 23,27,4,n,z|`	depth: drawn after layer z (default 1, over the bitmap)
`VDU 23,27,5,n,m|`	sprite n shows sprite m's picture
`PLOT &ED,x,y`	show the selected sprite, bottom-left at x, y

A placed sprite is a register write: moving sixty-four of them every frame costs the interpreter nothing but the PLOTs. `SPRITES.BBC`, `INVADERS.BBC` and `TUNNEL.BBC` in `/BBCBASIC` are the demonstrations (the last one moves nothing at all — it cycles the palette with `VDU 19`).

4.3 The terminal: JIM

The console you see when the Tube is running is not the ROM's own terminal but *JIM*, at `$DA00` — the Beeb's third I/O page, given a job here. JIM is a VT100 with the ANSI colours and the VT220 editing sequences (insert and delete character and line, erase character, scroll regions, origin mode, DEC line drawing, cursor-position and identity reports), built into the machine the way a real 8-bit computer got a serious terminal: as a card, not a program. It draws on the same text screen as the ROM console, inside the window the ROM gives it, so the two share one screen and one cursor. Anything that needs a terminal writes its byte stream to `$DA00` and reads its answers from `$DA02`; keys pushed through `$DA03` come back as the VT sequences the far end expects (arrows, Home/End, PgUp/PgDn, F1–F12, Delete as `$7F`). The register map is in `core/term.h`.



Turbo Pascal 3 (`H:` user 3) on JIM: the menu bar in reverse video, the compiler's own screen handling on a VT100 that is a chip.

For CP/M this settles the question every program asks at install time: tell WordStar's `WSCHANGE`, Turbo Pascal's `TINST`, ZDE and the rest that the terminal is a *VT100* (or *ANSI*: the same sequences). Turbo Pascal 3 on `H:` user 3 comes up with its menu already right; WordStar 4 on `E:` wants one pass of `WSCHANGE`. BBC BASIC's console edition speaks the same language natively, so `COLOUR`, `CLS` and `PRINT TAB(x,y)` land where they should. Bytes `$80`–`$FF` are drawn as CP437 glyphs, which is what BBS ANSI art is made of.

4.4 Star commands

A line starting with * goes to the machine: `*DIR`, `*CD`, any shell command — and BBC BASIC's own file words (`LOAD`, `SAVE`) read and write the machine's filesystem directly, because the co-processor lives inside `fs/` too. The Tube starts in the shell's current directory, so `CD BBCBASIC` then

BBC lets `LOAD "KALEID.BBC"` work with no directory prefix; the machine's filesystem is the co-processor's whole world — `fs/` is shown as `/`, the host tree above it hidden, and `*CD ..` stops at the root.

Edges, honestly: `POINT(` and `TINT` are not implemented; `GCOL` modes 1–4 draw plain; `VDU 5` text prints as text. On the Pi both co-processors run on core 3, the second processor.

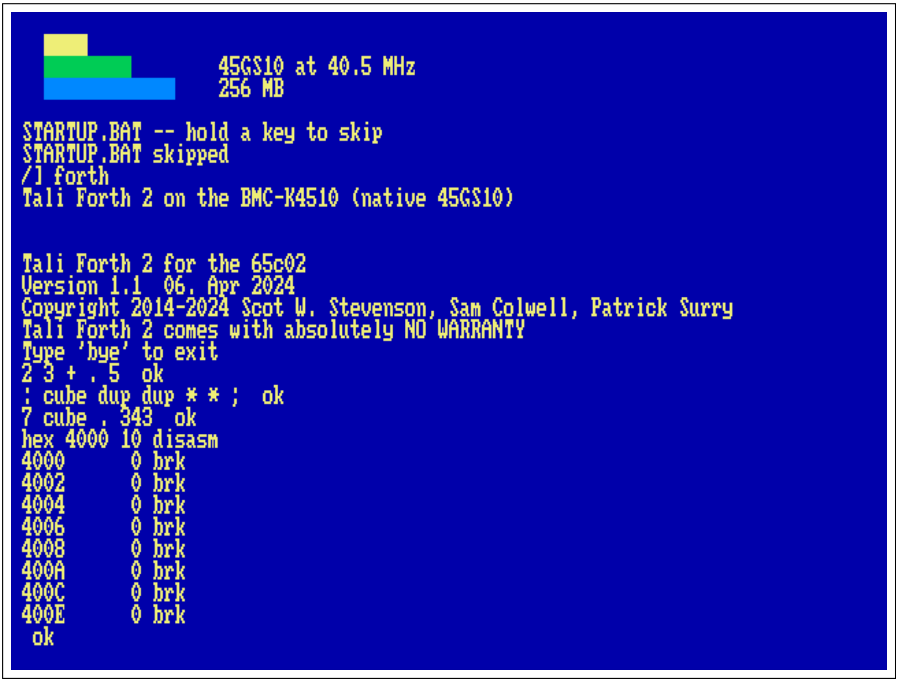
Chapter 5

Forth

The machine's third language is native: no Tube, no co-processor, just 45GS10 machine code loaded at `$4000`. It is Tali Forth 2 — a public-domain, ANS-flavoured Forth written for the 65C02, which this CPU speaks as a strict superset.

```
FORTH
```

```
2 3 + . 5  ok
: cube dup dup * * ;  ok
7 cube . 343  ok
```



A Forth session: the banner, arithmetic, a new word, and the kernel disassembling itself.

5.1 The machine at your fingertips

Forth's oldest habit is poking hardware, and this machine is one large, friendly memory map. `C@` and `C!` reach every register of Chapter 10 directly:

```
hex
D000 C@ .          read a VICKY register
D5E0 80 swap C!    HUSH, by hand: silence the sequencer
```

5.2 The assembler

Tali brings an interactive 65C02 assembler and a disassembler, kept in because on this machine they are the point:

```
4000 10 disasm       disassemble the Forth kernel itself
assembler-wordlist >order
here  2 lda.#  0 sta.z  drop
```

`DISASM` is strictly 65C02: at a 45GS10-only opcode it prints a polite `?` and moves on.

5.3 Manners

The dictionary has about 31.7 KB free for your words. `BYE` returns to the shell with the screen and the session exactly as you left them.

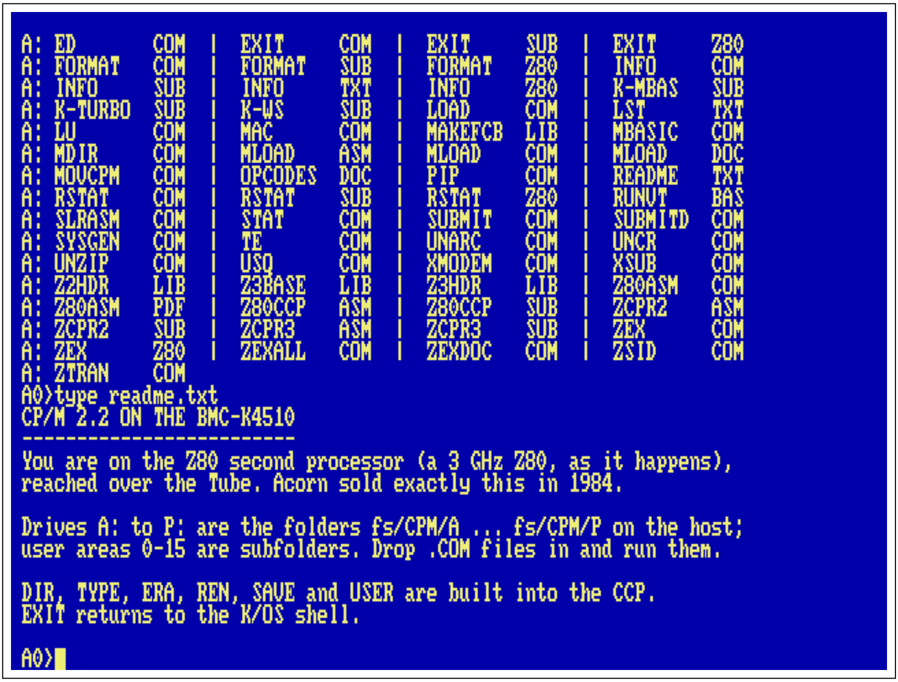
> No file words yet: Forth source lives in `/FORTH` but must be typed. An `INCLUDED` that reads through the machine's storage device is on the list.

Chapter 6

CP/M: the Z80 Second Processor

In 1984 Acorn sold a Z80 Second Processor for the BBC Micro; its purpose was to run CP/M, the operating system that owned business computing before the IBM PC. It sat on the Tube. So does ours. `CPM` at the shell boots CP/M 2.2 on a Z80 co-processor (RunCPM, on the host), and the `A0>` prompt that defined a decade appears. The boot banner cheerfully measures the Z80 in *gigahertz* — read that number knowingly: it is the emulated Z80's speed *on whatever host you run the emulator on* (a laptop, in this book's screenshots), so it varies machine to machine. For scale, the real thing shipped at 4 MHz.

```
CPM
```



CP/M 2.2 over the Tube: the CCP's `DIR` and `TYPE`, on drive A.

6.1 Drives are folders

Drives `A:` to `P:` are the host folders `fs/CPM/A` through `fs/CPM/P`; CP/M's user areas 0–15 are numbered subfolders. Drop any `.COM` file into `fs/CPM/A/0` and run it by name. `DIR`, `TYPE`, `ERA`, `REN`, `SAVE` and `USER` are built into the CCP; `EXIT` hands the console back to the K/OS shell.

Three of the drives are shortcuts rather than folders of their own: `K:` is the machine's own filesystem, so CP/M and K/OS can hand files to each other in both directions; `P:` is Turbo Pascal and `D:` is WordStar, each pointing straight at the user area the program lives in.

6.2 Starting a program from the shell

`CPM` on its own gives you the `A0>` prompt. `CPM command` runs a command at boot instead, through the CCP's `AUTOEXEC.TXT`. One line is all the CCP reads — but a name with no `.COM` to match runs the `.SUB` of that name, and that may be as long as you like. `/CPM/A/0/K-TURBO.SUB` is the pattern:

```
P:
TURBO
EXIT
```

Ending in `EXIT` is what makes the round trip whole: quitting a CP/M program only drops you to the CCP, and the `EXIT` carries you the rest of the way back to the K/OS prompt. With that in place `ALIAS TURBO CPM K-TURBO` makes `TURBO` a word you can type at the shell; `/SYSTEM/STARTUP.SAMPLE` defines that one, `WS` and `MBASIC`.

Why not `USER` in a submit. The obvious script is `H:`, `USER 3`, `TURBO` — and it dies on the second line. The submit in progress is a file, `$$$.SUB`, and it lives on A: user 0; `USER 3` walks away from it and it cannot be read any more. That is CP/M's own behaviour and not RunCPM's doing. Pointing a drive at the user area, as `P:` and `D:` do, keeps `USER` out of it.

6.3 Typing a CP/M program's name directly

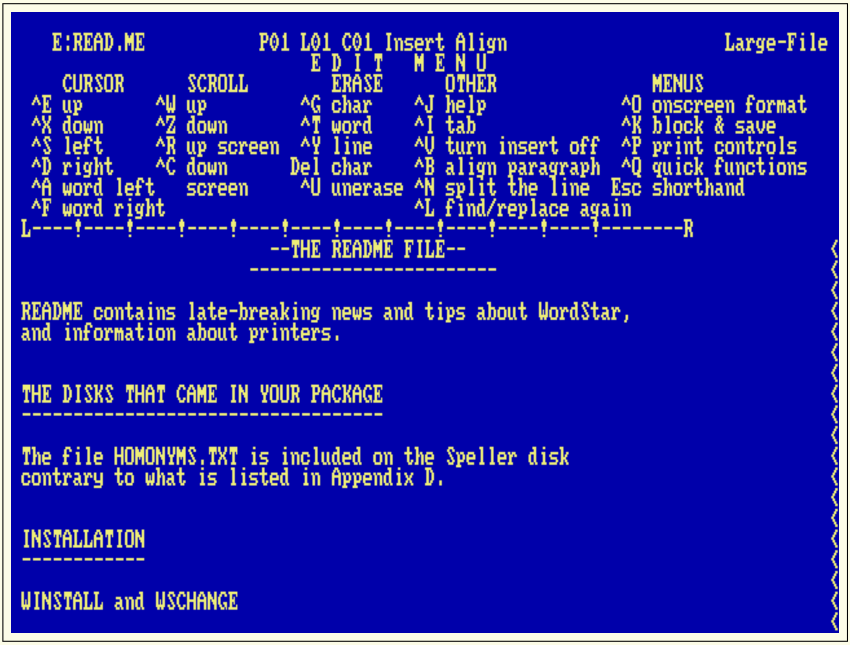
The shell can be told to treat an unknown word as a CP/M program: F7 → Shell → *CP/M .COM by name*. With it on, `STAT` at the /] prompt starts CP/M, runs `STAT.COM` from A: and comes back.

It is off to begin with, and deliberately so — `D` typed for `DIR` should not start a Z80 program. It changes only the guess: a command naming `CPM` outright, an alias among them, works either way. There is nothing in a `.prg` to tell the two apart, incidentally — its header is a load address and a run address, and a `.COM` begins with Z80 code that would read as a perfectly plausible one — so the extension is what decides.

6.4 Software

The RunCPM project ships a complete system disk (`DISK/A0.zip` in its repository): Digital Research's own `ASM`, `MAC`, `DDT`, `ZSID`, `STAT`, `PIP` and `ED`, the `SUBMIT` batch system, Microsoft BASIC, a Z80 assembler with its manual, XMODEM, and the source code of the BDOS and CCP — buildable on the machine itself. One `unzip A0.zip -d fs/CPM/` installs the lot. Beyond that lies the whole surviving CP/M software world: WordStar, Turbo Pascal, dBase II, Zork, one archive away.

Edges, honestly: line-mode programs (assemblers, compilers, MBASIC, adventures) work today, and so does the full-screen software: the console is JIM, a VT100 in hardware (Chapter 4). WordStar 4 on `E:` user 3 comes installed for "ANSI Standard" at 79×29 — `WS` and you are editing; Turbo Pascal 3 on `H:` user 3 needs nothing either. A program that asks its terminal type at install time wants "VT100" or "ANSI".



WordStar 4 editing its own `READ.ME`: the edit menu, the ruler, the text — 79×29 on JIM.

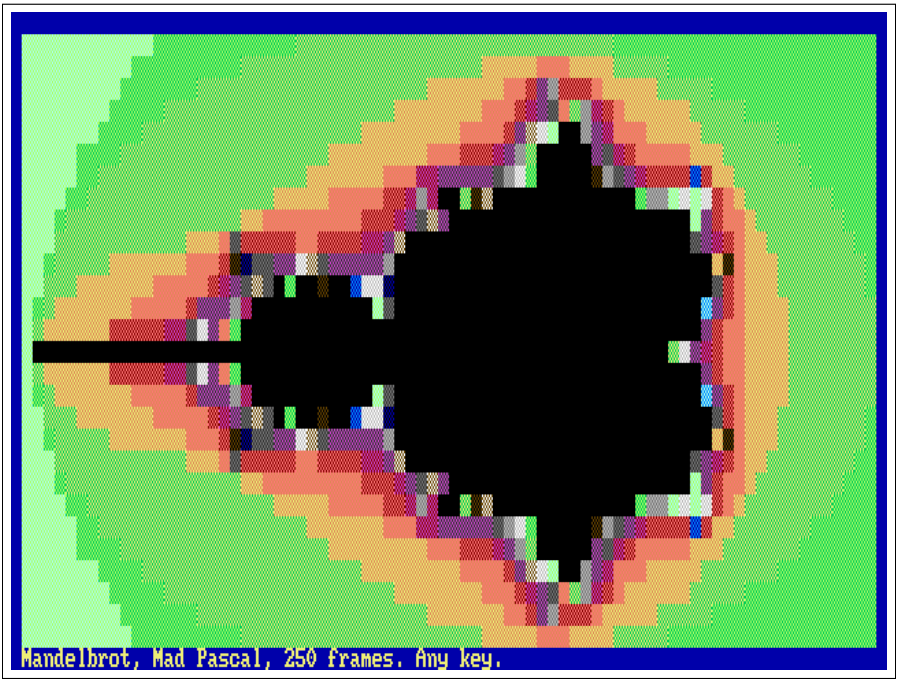
Desktop host only, like the rest of the Tube.

Chapter 7

Pascal

The machine has two Pascals, and they are different animals. Turbo Pascal 3 runs on the Tube under CP/M (Chapter 6): an on-machine compiler, a Z80 program, its own world. *Mad Pascal* is the other kind — a cross-compiler, like cc65 the ROM is built with: you write on the desktop, `mp` compiles to 6502 assembly, MADS assembles it, and the result is a `.prg` the shell's `RUN` loads like any other program. It is Tomasz Biela's compiler (MIT), a Turbo-Pascal-flavoured language with 8-, 16- and 32-bit integers, fixed and floating point, strings, records, pointers, units and inline assembly, made for the Atari and since taught the C64, the X16 and the Neo6502. The K4510 is its newest target.

```
PMANDEL
```



PMANDEL: the Mandelbrot set in text, sixteen colours through JIM, in 250 frames.

That is demo/pas/pmandel.pas: the Mandelbrot set, 78 by 28 cells in the machine's sixteen colours, in a little over four seconds at 40.5 MHz — fixed point, 32-bit integers, forty lines of Pascal. PSIEVE is the classic sieve, ten passes timed by the frame counter; HELLO prints, shows the colours, and echoes what you typed after its name.

7.1 The target

Everything a Pascal program needs from the machine is in pascal/ of the repository, laid out as it lives inside a Mad-Pascal checkout: the runtime base (base/rtl6502_k4510.asm and base/k4510/), where @putchar writes to JIM, the terminal (Chapter 4); the SYSTEM and CRT units' machine halves (lib/*_k4510.inc); and a k4510 unit. The consequences for the programmer:

- `Write` and `WriteLn` go through JIM, so the CRT unit is the real thing: `GotoXY`, `TextColor` and `TextBackground` (the palette's constants: `BLUE`, `YELLOW`, `LIGHT_GREEN...`), `ClrScr`, `ClrEol`, `InsLine/DelLine`, `WhereX/WhereY`, `CursorOn/CursorOff`, `ReadKey` and `KeyPressed` on the keyboard device, `Delay` and `Pause` on the frame counter, `Sound` on SID 0. `TextMode(0)` puts the ROM's screen back.

- `ParamCount` and `ParamStr` come from the shell line (`RUN HELLO one two`, or just `HELLO one two`); `ParamStr(0)` is the whole tail.

- `uses k4510` gives every chip as a typed variable at its address — `VICKY[reg]`, `SIDREG[]`, `DMA_SRC`, `FS_CMD`, `SYS_FRAMES`, `MATH_F[]`, `TERM_CX` and the rest — plus `FarPeek/FarPoke` into the 256 MB through the 45GS02's flat addressing, `DmaCopy/DmaFill`, `Shell('DIR')`, `LoadFile/SaveFile` through the ROM, `WaitVBlank`, and `TermWrite` for raw escape sequences.

- Memory: code from `$0800`, the program's own; zero page `$22--$3F` for the compiler's registers and `$64--$A3` for its expression stack (the ROM keeps `$02--$21` and `$F0--$F9`); `$0300` the string buffer. Programs return to the shell with the ROM's state intact.

7.2 Building

On the desktop, with FPC and a Mad-Pascal checkout (the installer patches its target table and rebuilds `mp`). It looks for the checkout in `~/Projects/neo6502_dev/Mad-Pascal` unless `MP_DIR` says otherwise:

```
make pascal-install    # once; MP_DIR=... to point elsewhere
make pascal            # demo/pas/*.pas -> fs/PRG/*.prg
```

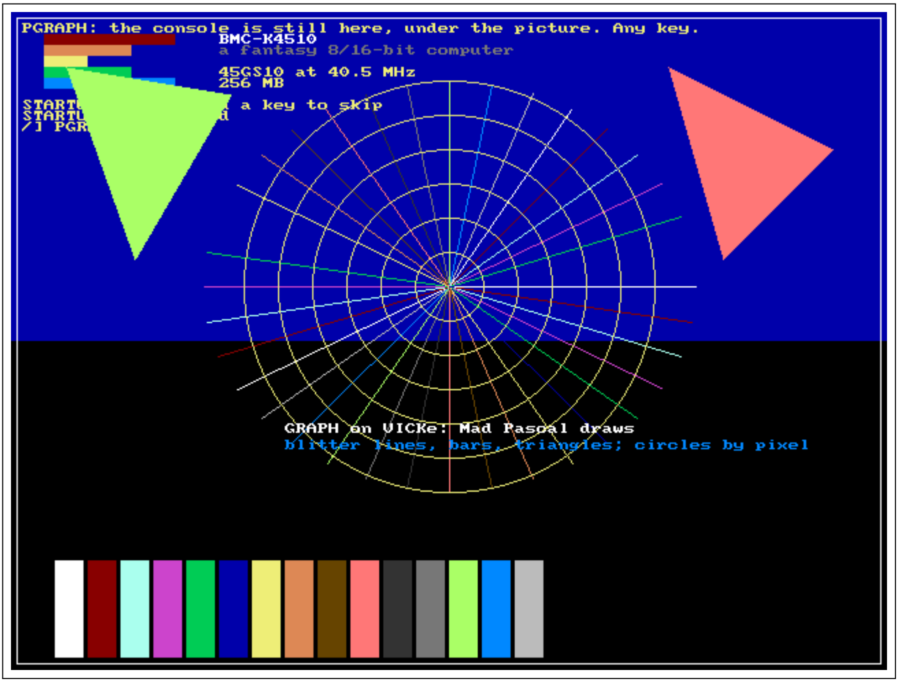
A new program is a file in `demo/pas/`; `make` finds it. The compiled `.prg` files are committed, so a machine without the toolchain still runs them, and the test suite (`test/pastest.sh`) runs `HELLO` and `PSIEVE` through the ROM.

7.3 Floating point on the MATH unit

single (IEEE-754, 32-bit) does not run in software here. The runtime's add, subtract, multiply, divide, compare, Trunc, Round and Frac are the MATH unit at $D700 (Chapter 10): each is a few register moves and one write to FOP, and the unit answers in the next cycle. PFLOAT times five thousand rounds of multiply, divide, add and subtract: 5 frames on the unit, 24 with the software library (assemble with mads -d:SOFTFLOAT=1 to get it back for comparison). The transcendentals are in the k4510 unit — MathSqrt, MathSin, MathCos, MathTan, MathAtan, MathAtan2, MathExp, MathLn, MathPow, MathFloor — one register write each, alongside the SYSTEM unit's own polynomial Sqrt, Sin and Cos, which still work and are still slower.

7.4 Graphics on VICKY

uses graph draws on VICKY's layer 1: a 640×480 8-bit bitmap at $200000 — the one BBC BASIC's ULA uses too — over the console, which stays visible wherever the picture is transparent (colour 0), as in a BBC MODE. InitGraph(0) switches the video to 480 lines and clears the bitmap; CloseGraph puts the console's mode back. PutPixel and GetPixel go through the 45GS02's flat addressing (the MEGA65 multiplier does the row arithmetic); Line, LineTo and the K4510 additions DrawLine, FillBar and FillTriangle are the blitter's LINE, FILL and TRIANGLE operations — one register write each; Circle, Ellipse, Rectangle and the rest of Mad Pascal's generic GRAPH build on those; OutTextXY writes with the console's font. PGRAPH is the demonstration.



PGRAPH: bars, a star and triangles by the blitter, circles by pixel, text from the ROM's font — and the console's own line still on top.

Note that SYSTEM's `Sqrt/Sin/Cos/ArcTan/Exp/Ln` on `single` also run on the MATH unit now: the installer patches `system.pas` under `{$ifdef k4510}`.

Chapter 8

The Editors

Two of them, because they answer different questions. EDIT is the one to reach for when a file wants a line changed and you want to be out again in ten seconds. VI is the one for a file too big to think about, and it is modal, so it expects you to have met vi before.

Both draw through JIM, the VT100 in hardware (Chapter 4), and both take their keys raw from the ROM — an arrow arrives as one byte rather than an escape sequence to unpick. Neither needs the Tube, so both run on the Pi as well as the desktop.

8.1 EDIT

EDIT name opens a file, or starts an empty one if there is no such file yet. The whole text sits in memory below the program, so it holds about 22 KB — ample for a STARTUP.BAT, a .SUB, a BASIC listing or a letter.

```
arrows Home End PgUp PgDn   move
Backspace Delete            rub out
Enter                       split the line
Ctrl-O  or  Ctrl-S          save
Ctrl-X                      leave
```

The bar along the bottom carries the name, a * while there are unsaved changes, where the cursor is, and those last two keys, so there is nothing to remember. Backspace at the start of a line joins it to the one above. Long lines slide sideways rather than wrap. Ctrl-X on a changed file says so and waits: press it again to throw the changes away, or Ctrl-O to keep them.

8.2 VI

`VI name` is modal in the old way: the keys are commands until `i` (or `a`, `I`, `A`, `o`, `O`) puts you into insert, and Escape brings you back out.

```
h j k l  arrows      move           0  $       line ends
gg  G                top, bottom    PgUp PgDn  pages
i a I A              insert         o  O       open a line
x   dd               delete         Esc        leave insert
:w  :q  :q!  :wq  :x
```

`:q` on a changed file refuses and says so; `:q!` throws the changes away and `:wq` keeps them. `:w name` writes somewhere else. Lines past the end of the file are marked ~ down the left, as they should be.

Where the file lives. Nothing of it is in the 64 KB. Every line is a 256-byte slot in far memory — a length and up to 255 characters — and only the line under the cursor is brought down, fetched when the cursor arrives and written back when it leaves. So the ceiling is far memory rather than the address space: 32000 lines, and `LOAD` and `SAVE` go straight to and from a flat copy out there without the file ever passing through the CPU's view.

That is the whole design, and it is deliberately the *only* design: there is no small-file case kept separately to disagree with the large one at exactly the wrong moment. It costs nothing because the DMA engine copies with `memmove` semantics, so opening or closing a line is a single transfer of everything below it however long the file is. A 1.28 MB file of 20000 lines — twenty times the whole address space — opens, edits at the far end and saves back byte for byte.

8.3 Editing without losing what you were doing

A program loaded from the shell lands on top of whatever was in memory, so running an editor from inside a BASIC would ordinarily destroy the program you were writing. `SWAP` (Chapter 2) is the way round it:

```
*SWAP EDIT DEMO.BAS
```

from EhBASIC puts the machine away, runs the editor on a clean one, and gives yours back — program, variables, screen and all — when the editor exits.

Part II

Programmer's Guide

Chapter 9

Memory

9.1 The 64 KB view and the 256 MB behind it

The 45GS10's program counter is 16 bits: code executes inside a 64 KB window. The 256 MB of physical memory is reached three ways:

- **Flat pointers**: `LDA [ptr],Z` and the Q forms read and write any byte of the 256 MB directly. Data never needs banking.
- **Bank registers** (`$D600`, one per 8 KB block): write a 28-bit base and that block of the window shows that memory. Bytes 0–2 set the base; byte 3 switches (bit 7 = off).
- **DMA** (`$D200`): copy, fill, swap, line, triangle — instant, physical addresses.

9.2 The CPU view, unmapped

`$0000-$03FF`	system	zero page, stack, vectors, low system state
`$0800-$CFFF`	user	50 KB for programs, ROM-free at launch
`$A000-$BFFF`	ROM	*the sideways window; RAM underneath*
`$C000-$CFFF`	ROM	*RAM underneath, bankable*
`$D000-$DFFF`	I/O	always I/O, whatever is banked
`$E000-$FEFF`	ROM	*RAM underneath, bankable*
`$FF00-$FFFF`	stub	always ROM: system-call stub and vectors

9.3 RAM under the ROM

`rom_out()` (or three pokes to the bank registers) banks blocks 5–7 onto the RAM beneath the ROM: a program then owns `$0800-$CFFF` + `$E000-$FEFF` = 57 KB. Every system call still works — calls go through the `$FF00` stub page, which banks the ROM in and out around them. `RUN romout` demonstrates the whole mechanism on screen.

9.4 Sideways ROM

The operating system is bigger than its half of the 64 KB — so, like the BBC Micro before it, the machine grew *sideways*. The ROM file is a 24 KB base image plus appended 8 KB banks; cold commands live in the `$A000-$BFFF` window (bank 0 is the base image itself, `INFO` and `TIME` already run from bank 1), and the shell pages the right bank in around each call. System calls work throughout: every call already passes through the always-ROM stub page at `$FF00`, which banks the ROM view in and out. Up to sixteen banks — 128 KB of operating system — fit in the scheme.

9.5 RAM under the I/O page

A `MAP` of block 6 hides `$D000-$DFFF` and exposes RAM: with the ROM also banked away a program owns a contiguous field from `$0800` to `$FEFF` — 61.75 KB of the 64. The trick that makes it safe is that `MAP` is an *instruction*, not a register: the program that hid the I/O can always ask for it back, so there is no way to lock yourself out. (The far-call gate is unreachable while block 6 is mapped; unmap before far calls.)

9.6 Programs bigger than the window

The `K4SG` executable format loads segments to any physical address and the far-call gate (`$DF00`) calls between them: a plain `JSR` into slot n banks descriptor n's code in, and the callee's `RTS` banks it back out. `RUN segdemo` shows two overlays sharing one address.

Chapter 10

The I/O Page

One page, `$D000-$DFFF`, always visible. The authoritative register-level reference is `core/io.h` in the source tree; this chapter will be generated from it, the way the Neo6502 handbook generates its keyword reference from the firmware source. Until then, the map:

`$D000`	VICKY	video: layers, palette, blitter, sprites, SHEILA
`$D100`	input	keyboard FIFO, status, peek, break-pending
`$D200`	DMA	copy / fill / swap / line / triangle
`$D300`	storage	the host filesystem: open, read, dir, chdir...
`$D400`	SID 0-3	four chips, 32 bytes apart
`$D500`	system	clock, RTC, frame counter, DUMP, SID crystal
`$D600`	banks	the eight bank registers, masks at `$D620`
`$D700`	MATH	IEEE floats, transcendentals, math lists, mul/div
`$D800`	Tube	the co-processor: status, byte in, byte out, start/stop
`$D900`	N:	the network: four channels, tcp://, http(s):// and tnfs:// (see `core/net.h`)
`$DA00`	JIM	the terminal: a VT100/ANSI in hardware, drawing on the console's screen (see `core/term.h`)
`$DF00`	far gate	call slots, descriptor table pointer, depth

Disclaimer

The BMC-K4510 is a hobby project: a computer that does not exist, built for the pleasure of building it. It is offered to you as a gift, and it comes with exactly the guarantees a gift comes with — none.

In the words of the licence it is distributed under (page 48), and meaning every one of them:

> **THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW.** EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.
> **IN NO EVENT** WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM.

In plainer words: nobody promises this machine works, that it will keep working, or that anything it writes to your disk is safe. Keep backups. Do not fly aeroplanes with it, run hospitals on it, or trust it with money. It is a toy, and it is allowed to be wrong.

The same goes for this book: where it and the machine disagree, the machine is right. Nothing here is endorsed by or affiliated with Acorn, Commodore, Microsoft, Digital Research, the MEGA65 project, the Raspberry Pi Foundation, or anyone else gratefully named in these pages. The mistakes are the author's alone.

Constructive comments can be left at the repository's issues page:
`https://github.com/mlongval/bmc-k4510/issues`

All complaints, criticisms and negativity can be addressed to
`/dev/null`

Thank You

The BMC-K4510 is a machine that never existed. Almost everything in it that *does* exist was written by somebody else first, and given away. This chapter is the thanks; the Licences chapter (page 48) is the paperwork.

The heart of the machine

Gábor Lénárt (LGB) wrote the 45GS02 CPU core in Xemu, the MEGA65 emulator. It runs here unchanged — not a line altered, not a bug fixed — and every instruction this book describes is his code executing. There would be no K4510 without it.

Dag Lem wrote reSID, the SID emulation that has been the reference for thirty years. The K4510's four sound chips are four copies of it (the 6581 model, as shipped in VICE 3.3).

The VICE team, whose decades of emulation scholarship — documentation, test programs, arguments settled in code — this project consulted at nearly every turn.

The tongues

Lee Davison (1966–2013) wrote EhBASIC, the machine's first BASIC and still the one that comes up in ROM. He is not here to be asked about the graphics and sound words bolted onto it; the machine carries his name in its `README` and its startup banner instead.

Richard T. Russell wrote BBC BASIC, and has kept writing it for forty years. The console edition (BBCTTY) is what runs on the Tube co-processor in Chapter 4. The name "BBC BASIC" is used here *by his permission*, which is a courtesy this project does not take lightly.

Marcelo Dantas (“Mockba the Borg”) wrote RunCPM, which is the whole of Chapter 6: a complete CP/M 2.2 with its own CCP, vendored here unmodified and simply handed a Z80’s worth of address space.

Scot W. Stevenson, **Sam Colwell** and **Patrick Surry** wrote Tali Forth 2 and put it in the public domain — a Forth written to be read, which is why Chapter 5 can be honest about how it works.

Tomasz Biela (“tebe”) wrote Mad Pascal and MADS; the K4510 target in `pascal/` is a guest in his compiler. **Wojciech Bociański** (“bocianu”), whose Neo6502 target showed how such a guest should behave.

Steve Wozniak wrote Wozmon in 1976 in 256 bytes, and the monitor in this machine is still recognisably it.

Michael Steil maintains msbasic, and **Microsoft** released 6502 BASIC 1.1 under the MIT licence in 2025 — between them, the machine’s next native BASIC.

Letters on the screen

Fonts are the part of a computer you look at longest, and clean ones with a clear pedigree are hard to come by.

The Linux kernel contributors — the 8x8 console font (`font_8x8.c`) that got the text mode on its feet and is still the default.

Paul Gardner-Stephen and **Roman Standzikowski** (FeralChild64) — the clean-room C64/C65 chargen from MEGA65 open-roms, and **Retrofan**, whose PXLfont travels with it by permission.

Ville-Matias Heikkilä (“Viznut”) — unscii, placed in the public domain.

Damian Vila — BESCII, the PETSCII spirit with a clean pedigree, CC0.

The bare metal

Randy Rossi wrote BMC64: VICE on a Raspberry Pi with no operating system under it, 50 Hz smooth scrolling and input latency measured in single frames. It is the direct reason this machine has a Pi port at all

— the route to the Pi was always meant to be BMC64's `emux_api` seam, and the bare-metal case it proved is what made the port thinkable. He also built the **VIC-II Kawari**, a modern drop-in VIC-II with video modes the original never had: the demonstration that you may extend an 8-bit machine's video chip and still be working inside the tradition rather than outside it. VICKY is a more reckless cousin of that idea. **minch** (aminch) has taken BMC64 over from him and carries it onto the newer Pis; the project is in good hands.

Rene Stange wrote Circle, the bare-metal C++ environment that is the entire reason a Raspberry Pi 3B+ can boot straight into this machine with no operating system under it. **Xalior** wrote circle-libsdl2, which let the desktop emulator move to the Pi essentially as written.

The workshop

The machine is built with **cc65** (compiler, assembler, linker and run-time for the whole system ROM), **64tass**, **NASM**, **GCC** and **GNU Make**; it shows itself to you through **SDL2**. This book is set in **XeLaTeX** with **Clear Sans** and **Iosevka**, and every screenshot in it was captured from the machine actually running — a picture that cannot be produced fails the build.

Heritage

Some debts are not code.

Acorn Computers — the Tube, the sideways-ROM model, the * command prefix, and the idea that a second processor should be a normal thing to own. The Beeb's ghost is all over this machine.

Commodore — the other half of its soul: PETSCII, the SID, and the line from the PET to the C65 that stopped one machine short of this one.

The MEGA65 project — for keeping the 45GS02 and the C65 dream alive in real silicon, and for open-roms.

Kim Lemon and the Lemoners — Lemon64 has been the C64's front door since 1998: the games database, the reviews and scans, the music, and a forum that actually answers. Twenty-eight years of one person's dedication to a machine's community, and a good half of the small facts this project needed came from there.

The Neo6502 handbook and the **MEGA65 User's Guide** — the two books this one is modelled on, down to the page size.

The High Voltage SID Collection and the composers in it, whose tunes are what `SIDPLAY` plays on a good evening. None of that music is distributed with this machine (see the Licences chapter); it is theirs.

> **And the machine's other author.** Most of the K4510's own code — VICKY, SHEILA, the DMA engine, the ROM, K/OS, the Tube ULA, the editors, the Pi port — was written in conversation with Claude (Anthropic's Claude Code), over several months of long sessions with a build log to prove it. The design decisions are the author's; a great deal of the typing was not.

And whoever is missing

This list was assembled by hand and is certainly incomplete. If your work is in this machine and your name is not on this page, that is an error and not a judgement — open an issue at `https://github.com/mlongval/bmc-k4510` and it will be fixed in the next printing.

Licences

The Thank You chapter (page 44) is the thanks; this is the record. The authoritative copy is `LICENSES.md` in the repository — if the two ever disagree, that file wins.

The project itself

The BMC-K4510 as a whole is distributed under the **GNU General Public License, version 2 or (at your option) any later version**. Everything written for this project — the machine, VICKY, SHEILA, the DMA engine, the MATH unit, the system ROM and K/OS, JIM, the Tube ULA, the network layer, the editors, the demos, the Pascal target and the Raspberry Pi port — is Copyright © 2026 Michael Longval and is released under those terms. The choice is not really a choice: the CPU core and reSID are GPL-2.0-or-later, and a work containing them must be too.

Code inside the machine

- **Xemu 65xx/45GS02 CPU core** — Gábor Lénárt. `core/xemu/`: `cpu65.c`, `cpu65.h`, `cpu65_mega65_timings.h`, `cpu65ce02_disasm_tables.c`, used unchanged. *GPL-2.0-or-later.*

- **reSID** — Dag Lem, as shipped in VICE 3.3. `core/resid/`. *GPL-2.0-or-later.*

- **BBC BASIC, console edition (BBCTTY)** — Richard T. Russell. `tube/`, vendored and altered (see `tube/ALTERED.md`). *zlib licence.* The name "BBC BASIC" is used by permission; that permission is personal to this project and is *not* transferable to forks or derivatives.

- **EhBASIC 2.22** — Lee Davison, in the ca65 form from jefftranter/6502. `basic/basic.asm`. *Free for non-commercial use only*; derivatives must carry the words "Derived from EhBASIC" — see `basic/`

`README-EhBASIC.txt`. It ships as a separate program (`fs/ehbasic.prg`) and is not linked with the GPL code.

- **RunCPM** — Marcelo Dantas. `cpm/src/`, vendored unmodified (`cpm/VENDORED-FROM.txt`). *MIT.*
- **Tali Forth 2** — Scot W. Stevenson, Sam Colwell, Patrick Surry. `forth/tali/`, vendored unmodified. *Public domain.*
- **Wozmon** — Steve Wozniak's 1976 monitor, reimplemented here for the 45GS10. Apple published the original listing; this machine's version is the project's own code under the project licence.
- **cc65 runtime** — the ROM and every `.prg` are linked against `none.lib`. *cc65's zlib-style licence.*
- **Microsoft 6502 BASIC 1.1** (planned, via mist64/msbasic) — *MIT*, released by Microsoft in 2025. Not yet in the machine.

Fonts

- **Linux kernel 8x8 console font** — `data/font8.bin`, built by `data/mkfont.py` from `lib/fonts/font_8x8.c`. *GPL-2.0.* This is the default text font.
- **MEGA65 open-roms chargen + PXLfont 2.3** — `data/fonts/openroms/`. Clean-room C64/C65 character shapes; PXLfont is Retrofan's, included with open-roms' recorded permission. *LGPL-3.0-or-later* — `8x8font.png` is shipped as the editable source, for LGPL compliance.
- **unscii-8** — Ville-Matias Heikkilä. `data/fonts/unscii/`; `font8-unscii.bin` is a generated drop-in alternative to `data/font8.bin`. *Public domain.*
- **BESCII v3** (Mono + source) — Damian Vila. `data/fonts/bescii/`. *CC0-1.0.*
- **Clear Sans** (Intel, *Apache-2.0*) and **Iosevka** (Belleve Invis, *SIL OFL 1.1*) — the two faces this book is set in. They are not part of the machine.

No Commodore ROMs here. Until 2026-08-23 the text font was derived from a Commodore 64 character ROM. That file and every derivative were removed from the repository *and from its history* before it was made public. If you own a C64 and want the real thing, drop your own `chargen.bin` into `/SYSTEM`: the machine will use it, and `.gitignore` will keep it out of the repository. That copy is yours, not ours to publish.

The Raspberry Pi build

Neither of these lives in the repository; `pi/Makefile` expects them beside the checkout.

- **Circle** — Rene Stange. *GPL-3.0.*
- **circle-libsdl2** — Xalior. *zlib*; its `sdl-app.ld` is *GPL-3.0*.

A `kernel8.img` built from these is therefore GPL-3.0 as a whole, which GPL-2.0-*or-later* code permits. The Raspberry Pi boot firmware in a distributed card image (`bootcode.bin`, `start.elf`, `fixup.dat`) is Broadcom's, redistributable with Raspberry Pi hardware under its own licence.

Build tools

Not distributed with the machine, but required to build it: GCC and GNU Make (*GPL*), cc65 (*zlib-style*), 64tass (*GPL-2.0*), NASM (*BSD-2-Clause*), SDL2 (*zlib*), Python 3 (*PSF*), and for this book XeLaTeX (*LPPL*). Mad Pascal and MADS (Tomasz Biela, *MIT*) are needed only if you compile Pascal; the K4510 target in `pascal/` is installed into your own Mad Pascal checkout.

What is deliberately not shipped

- **The CP/M system disk.** RunCPM's `DISK/A0.zip` — Digital Research's `ASM`, `MAC`, `DDT`, `STAT`, `SUBMIT` and third-party tools — installs into `fs/CPM/`

`A/0/` for your use but is not committed: its files have mixed provenance and this repository is public. The same goes for WordStar, Turbo Pascal 3 and MBASIC, which the `K-*.SUB` launchers know how to start but which you must supply yourself.

- **SID tunes.** `fs/SID` is a symbolic link to a local copy of the High Voltage SID Collection. The tunes are the composers' copyright, are used here only for listening, and are not part of this repository or of any release.
- **A Commodore character ROM** — see the note above.
- **Your `STARTUP.BAT`** — yours, not the repository's (copy `/SYSTEM/STARTUP.SAMPLE` to start one).

If you redistribute the machine

Ship the sources you built from, keep `LICENSES.md` and this chapter with them, keep `8x8font.png` beside the open-roms font, and remember two things that are easy to forget: EhBASIC is non-commercial-only, and the "BBC BASIC" name is licensed to this project and not to yours. Call your fork's BASIC something else.